

BOOK 08 (PART 1) — JAVA MULTITHREADING & CONCURRENCY

Foundations: Threads, Synchronization & the Java Memory Model (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 08 of 24 (Part 1 of 2) **Java**

Versions Covered: Core concurrency model (stable since early Java), `java.util.concurrent.atomic` (Java 5), `java.util.concurrent.locks` (Java 5), Java 21 references where relevant (virtual threads deferred to Part 2) **Prerequisites:** Book 01–03 (objects/memory model, JVM internals), Book 05 (Collections), Book 07 (CompletableFuture basics, functional interfaces) **Next:**

`08_Java_Multithreading_Concurrency_Part_2.md` (ExecutorService, higher-level utilities, virtual threads, production patterns)

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Multithreading Java లో అత్యంత **advanced మరియు అత్యంత frequently misunderstood** topic. ఒకే memory ని multiple threads ఏకకాలంలో access చేసేటప్పుడు వచ్చే bugs — race conditions, deadlocks — subtle గా, hard-to-reproduce గా ఉంటాయి. ఈ Part 1 లో **foundations** — threads ఎలా create అవుతాయి, synchronization ఎలా పనిచేస్తుంది, Java Memory Model ఏమిటో — లోతుగా నేర్చుకుంటాము. Part 2 లో higher-level utilities (ExecutorService, CountdownLatch, Virtual Threads) చూద్దాం.

English: Multithreading is Java's most advanced and most frequently misunderstood topic — bugs from multiple threads sharing memory (race conditions, deadlocks) are subtle and hard to reproduce. Part 1 covers the foundations: how threads are created, how synchronization actually works, and the Java Memory Model. Part 2 covers higher-level utilities (ExecutorService, CountdownLatch, Virtual Threads) and production patterns.

Learning Objectives (Part 1)

1. Distinguish process vs thread, and understand the full thread lifecycle.
 2. Create threads via `Thread`, `Runnable`, `Callable`, and `Future`.
 3. Understand race conditions and why they happen.
 4. Master `synchronized` — intrinsic locks, method vs block synchronization.
 5. Master `volatile` and the visibility guarantee it provides (and doesn't).
 6. Understand the Java Memory Model: happens-before, atomicity, visibility, ordering.
 7. Use atomic classes and understand Compare-And-Swap (CAS).
 8. Use `ReentrantLock`, `ReadWriteLock`, `StampedLock` and know when they beat `synchronized`.
 9. Recognize and prevent deadlock, livelock, and starvation.
 10. Design thread-safe classes using immutability and proper encapsulation.
-

Table of Contents (Part 1)

Ch.	Title
1	Process vs Thread & Thread Lifecycle
2	Creating Threads: Runnable, Callable, Future
3	Race Conditions & Thread Safety Basics
4	<code>synchronized</code> — Deep Dive
5	<code>volatile</code> & Visibility
6	The Java Memory Model — Happens-Before
7	Atomic Classes & Compare-And-Swap
8	Locks: ReentrantLock, ReadWriteLock, StampedLock
9	Deadlock, Livelock & Starvation
10	Immutability & Thread-Safe Design + Mini Project
—	Part 1 Revision Notes, Cheat Sheet, Interview Bank, Mastery Checklist

CHAPTER 1 — Process vs Thread & Thread Lifecycle

Telugu Explanation

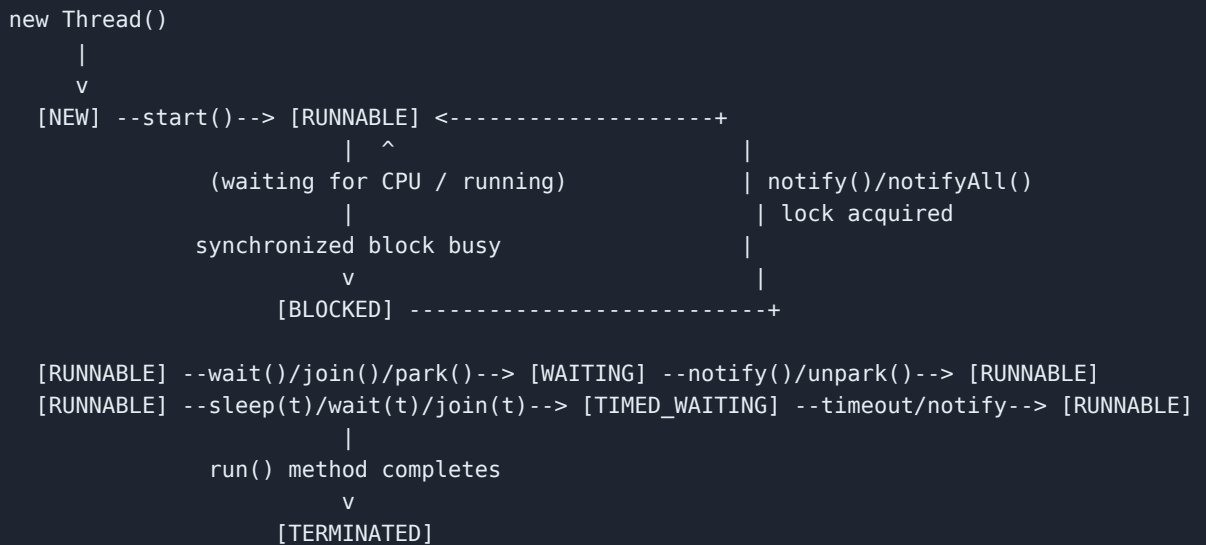
Process అంటే ఒక independently running program, తనదైన memory space తో (OS level isolation).

Thread అంటే ఒక process లోపల ఒక **lightweight execution unit** — ఒకే process లోని అన్ని threads **heap మరియు Method Area (Book 03) share చేస్తాయి**, కానీ ఒక్కొక్కటికీ తనదైన Stack, PC Register ఉంటుంది (Book 03, Ch.4). Thread కి 6 states ఉంటాయి: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.

Professional English Explanation

A **process** is an independently running program with its own OS-isolated memory space. A **thread** is a lightweight execution unit within a process — all threads of a process **share the Heap and Method Area** (Book 03), but each has its own Stack and PC Register (Book 03, Ch.4). A thread moves through 6 states: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`.

Diagram — Thread Lifecycle



Java Code

```
public class ThreadLifecycleDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            System.out.println("Worker running, state: " + Thread.currentThread().getState());
            try { Thread.sleep(200); } catch (InterruptedException ignored) {}
        }, "worker-thread");

        System.out.println("Before start(): " + worker.getState());           // NEW

        worker.start();
        Thread.sleep(50);                                                     // give it time
to reach sleep()
        System.out.println("While sleeping: " + worker.getState());           // TIMED_WAITING

        worker.join();                                                         // main waits
for worker to finish
        System.out.println("After join(): " + worker.getState());             // TERMINATED

        System.out.println("Available CPU cores: " +
Runtime.getRuntime().availableProcessors());
    }
}
```

Output (illustrative)

```
Before start(): NEW
Worker running, state: RUNNABLE
While sleeping: TIMED_WAITING
After join(): TERMINATED
Available CPU cores: 8
```

Internal Working

- Calling `run()` directly (instead of `start()`) does **not** create a new thread at all — it just executes `run()`'s code synchronously on the calling thread, a very common beginner mistake that silently produces "working" but entirely non-concurrent code.
- `RUNNABLE` in Java's model actually covers both "currently executing on a CPU" and "ready and waiting for CPU time" — the JVM doesn't expose a separate "ready" state; the OS scheduler decides which `RUNNABLE` thread actually gets CPU time at any instant.
- `BLOCKED` specifically means a thread is waiting to acquire a `synchronized` lock (Ch.4) held by another thread; `WAITING` / `TIMED_WAITING` mean the thread voluntarily gave up execution (via `wait()`, `join()`, `sleep()`, `park()`) rather than contending for a lock.

Real-World Example

Telugu: Web server ఒక్కో incoming request కి (traditional model లో) ఒక thread కేటాయిస్తుంది — ఆ threads అన్నీ ఒకే process యొక్క heap ని share చేస్తాయి (ఉదా. shared cache), కానీ ఒక్కొక్కటికీ independent request-processing state (stack) ఉంటుంది. English: A traditional web server assigns one thread per incoming request — all those threads share the same process's heap (e.g., a shared cache), but

each has independent per-request processing state on its own stack, which is exactly the shared-heap/private-stack model this chapter establishes.

Interview Answer

"A process is an isolated running program; a thread is a lightweight unit of execution within a process, sharing the process's heap and method area but with its own stack and PC register. A thread moves through NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED states — BLOCKED specifically means waiting on a synchronized lock, while WAITING/TIMED_WAITING mean voluntarily yielding via wait/join/sleep."

Cross Questions

- Q: What happens if you call `run()` instead of `start()`? → A: No new thread is created — the code runs synchronously on the calling thread, defeating the entire purpose of using a `Thread`.
- Q: Can a `TERMINATED` thread be restarted via `start()` again? → A: No — calling `start()` on an already-started (or terminated) thread throws `IllegalThreadStateException`; a new `Thread` object must be created.
- Q: What's the difference between `BLOCKED` and `WAITING`? → A: `BLOCKED` is specifically contention for a `synchronized` lock; `WAITING` is voluntary suspension via `wait()` / `join()` / `park()` with no timeout.

Tricky Questions

- Q: Does `Thread.sleep()` release any locks the thread currently holds? → A: No — `sleep()` only pauses execution; it does NOT release any `synchronized` locks held by that thread, unlike `wait()` (Ch.4), which does release the lock it's called while holding.
- Q: Is `RUNNABLE` the same as "currently running on a CPU core"? → A: Not necessarily — `RUNNABLE` means eligible to run; the actual CPU-running subset is an OS-scheduler-level detail the JVM doesn't distinguish in its own state enum.

Coding Exercise

L1: Create a thread, print its state before `start()`, during execution, and after `join()`. **L2:** Reproduce the "calling `run()` instead of `start()`" mistake and observe that no real concurrency occurs (same thread name printed). **L3:** Create two threads and use `join()` to make `main()` wait for both before printing a final message. **L4 (Interview):** Draw the full thread lifecycle diagram from memory. **L5 (Senior):** Explain, using the shared-heap/private-stack model, why a race condition (Ch.3) can occur on shared heap objects but never on local stack variables. **L6 (Mastery):** Explain, from memory, the distinction between `BLOCKED` and `WAITING` and give a code scenario producing each.

CHAPTER 2 — Creating Threads: Runnable, Callable, Future

Telugu Explanation

Thread create చేయడానికి రెండు classic పద్ధతులు: `Thread` class ని extend చేయడం (discouraged — single inheritance waste అవుతుంది), లేదా `Runnable` ని implement చేసి `Thread` కి పంపడం (preferred — composition, Book 02 Ch.12). `Runnable.run()` value return చేయదు, exception throw చేయలేదు. `Callable<V>` (Java 5+) value return చేస్తుంది, checked exception throw చేయగలదు — `ExecutorService` (Part 2) తో వాడతారు, `Future<V>` ద్వారా result తీసుకుంటారు.

Professional English Explanation

Two classic ways to create a thread: extend `Thread` (discouraged — wastes Java's single class inheritance, Book 02 Ch.4) or implement `Runnable` and pass it to a `Thread` (preferred — composition over inheritance, Book 02 Ch.12/13). `Runnable.run()` returns nothing and cannot throw a checked exception. `Callable<V>` (Java 5+) returns a value and can throw a checked exception — used with `ExecutorService` (Part 2), retrieving its result via a `Future<V>`.

Java Code

```
import java.util.concurrent.*;

public class CreatingThreadsDemo {

    static class ExtendsThreadApproach extends Thread {                // discouraged
        @Override public void run() { System.out.println("Running via Thread subclass"); }
    }

    static class RunnableApproach implements Runnable {                // preferred -
composition
        @Override public void run() { System.out.println("Running via Runnable"); }
    }

    public static void main(String[] args) throws Exception {
        new ExtendsThreadApproach().start();
        new Thread(new RunnableApproach()).start();
        new Thread(() -> System.out.println("Running via lambda Runnable (Book 07)").start());

        // Callable + Future: returns a value, can throw checked exceptions
        Callable<Integer> task = () -> {
            Thread.sleep(100);
            return 42;
        };

        ExecutorService executor = Executors.newSingleThreadExecutor();    // preview - full
detail in Part 2
        Future<Integer> future = executor.submit(task);

        System.out.println("Main continues immediately, not blocked...");
        Integer result = future.get();                                     // .get() BLOCKS
until the result is ready
        System.out.println("Callable result: " + result);

        executor.shutdown();

        // Future also reports task state
        Future<Integer> anotherFuture = executor.submit(task);
        System.out.println("isDone right after submit? " + anotherFuture.isDone());    // likely
false
    }
}
```

Output (illustrative — thread scheduling order may vary)

```
Running via Thread subclass
Running via Runnable
Running via lambda Runnable (Book 07)
Main continues immediately, not blocked...
Callable result: 42
isDone right after submit? false
```

Internal Working

- Extending `Thread` uses up Java's single-inheritance slot (Book 02, Ch.4) unnecessarily and conflates "being a thread" with "having a task to run" — `Runnable` / `Callable` cleanly separate the *task* from the *execution mechanism*, which is also exactly what makes them reusable with `ExecutorService`'s thread pools (Part 2) instead of creating a raw `Thread` for every single task.
- `Future.get()` **blocks** the calling thread until the task completes (or throws the original exception wrapped in `ExecutionException`) — it is the synchronous "give me the result now, wait if necessary" bridge back from asynchronous execution, conceptually similar to `CompletableFuture.get()` (Book 07, Ch.11), which in fact implements `Future`.
- `Future` also exposes `isDone()`, `isCancelled()`, and `cancel(boolean mayInterruptIfRunning)` for inspecting/controlling a still-running task without blocking.

Real-World Example

Telugu: Real backend services లో `new Thread()` directly create చేయడం చాలా అరుదు — production code దాదాపు ఎప్పుడూ `ExecutorService` (Part 2) + `Runnable` / `Callable` వాడుతుంది, thread lifecycle management, pooling, resource limits కోసం. English: Directly creating raw `Thread` objects is rare in real production backend code — almost everything goes through an `ExecutorService` (Part 2) submitting `Runnable` / `Callable` tasks, which manages thread lifecycle, pooling, and resource limits far more safely and efficiently than manual thread management.

Interview Answer

"Threads can be created by extending `Thread` (discouraged, wastes single inheritance) or implementing `Runnable` / `Callable` and running them via a `Thread` or, in production, an `ExecutorService`. `Runnable` returns nothing and can't throw checked exceptions; `Callable<V>` returns a value and can throw checked exceptions, retrieved asynchronously via a `Future<V>`, whose `get()` blocks until the result is ready."

Cross Questions

- Q: Why is implementing `Runnable` preferred over extending `Thread`? → A: It preserves the single-inheritance slot for actual class hierarchy needs (composition over inheritance, Book 02), and cleanly separates the task definition from thread-management mechanics, enabling reuse with thread pools.
- Q: Can `Runnable`'s `run()` throw a checked exception? → A: No — its signature declares no `throws` clause; a checked exception inside must be caught and handled (or wrapped as unchecked, Book 04) within `run()` itself.
- Q: What does `Future.get()` do if the task threw an exception? → A: It throws `ExecutionException`, wrapping the original exception (accessible via `getCause()`), which the caller must handle.

Tricky Questions

- Q: If you call `future.get()` immediately after `executor.submit(task)`, does it necessarily block? → A: It blocks only if the task hasn't completed yet — if the task happened to already

finish (e.g., a trivially fast task on an idle pool), `get()` returns immediately without blocking at all.

- Q: Can `Future.cancel(true)` guarantee the task stops immediately? → A: No — it merely interrupts the executing thread (setting its interrupted status) if `mayInterruptIfRunning` is `true`; the task itself must cooperatively check for interruption (e.g., via `Thread.interrupted()` or by letting a blocking call like `sleep()` throw `InterruptedException`) to actually stop promptly.

Coding Exercise

L1: Create threads using all three approaches (extending `Thread`, `Runnable`, `lambda Runnable`) and run them. **L2:** Submit a `Callable<Integer>` to an `ExecutorService`, retrieve its result via `Future.get()`, and print the elapsed time. **L3:** Reproduce `ExecutionException` by throwing an exception inside a `Callable`, and unwrap the original cause via `getCause()`. **L4 (Interview):** Explain why implementing `Runnable` is preferred over extending `Thread`. **L5 (Senior):** Design a task-processing method accepting a generic `Callable<T>` and returning a `Future<T>`, explaining the composition benefit over hardcoding `Thread` subclasses per task type. **L6 (Mastery):** Explain, from memory, the difference between `Runnable` and `Callable`, and how `Future.get()` bridges asynchronous execution back to synchronous retrieval.

CHAPTER 3 — Race Conditions & Thread Safety Basics

Telugu Explanation

Race condition అంటే, multiple threads shared mutable state ని ఏకకాలంలో access చేసేటప్పుడు, execution timing మీద ఫలితం ఆధారపడి, unpredictable/wrong results రావడం. Classic example: `count++` — ఇది **atomic operation కాదు** — internally మూడు steps (read, increment, write) ఉంటాయి, రెండు threads మధ్యలో interleave అయితే updates కోల్పోతాయి (**lost update**).

Professional English Explanation

A race condition occurs when multiple threads access shared mutable state concurrently, and the result depends unpredictably on execution timing. The classic example: `count++` is **not atomic** — it involves three internal steps (read the current value, increment it, write it back); if two threads' steps interleave, an update can be **lost**.

Java Code — Reproducing a Race Condition

```
public class RaceConditionDemo {
    static int counter = 0; // shared mutable state - no protection

    static void increment() {
        counter++; // NOT atomic: read -> increment ->
write
    }

    public static void main(String[] args) throws InterruptedException {
        int numThreads = 10;
        int incrementsPerThread = 100_000;
        Thread[] threads = new Thread[numThreads];

        for (int i = 0; i < numThreads; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < incrementsPerThread; j++) increment();
            });
        }

        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join();

        int expected = numThreads * incrementsPerThread;
        System.out.println("Expected: " + expected);
        System.out.println("Actual: " + counter + " (likely LESS than expected due to lost
updates)");
    }
}
```

Output (illustrative — actual result is non-deterministic and varies each run)

```
Expected: 1000000
Actual: 974213 (likely LESS than expected due to lost updates)
```

Internal Working — Why `count++` Loses Updates

Thread A	Thread B	
read counter (value = 5)	read counter (value = 5)	<- BOTH read the SAME stale value
increment locally to 6	increment locally to 6	
write counter = 6	write counter = 6	<- one increment is LOST!
	(counter should be 7, but is 6)	

- `counter++` compiles to at least three separate bytecode instructions (`getfield / getstatic` , `iadd` , `putfield / putstatic`) — there is no atomicity guarantee across these steps for concurrent access, regardless of how "simple" the source-code line looks.
- This is called a **lost update**: two threads read the same value before either writes back, so one thread's increment is silently overwritten by the other's — the total count ends up lower than the true number of increments performed.
- Fixing this requires either `synchronized` (Ch.4), an atomic class (Ch.7), or a proper lock (Ch.8) — simply "hoping it works" or reducing thread count doesn't eliminate the underlying non-atomicity, it just makes the bug rarer and harder to detect.

Real-World Example

Telugu: E-commerce లో "available stock count" ని multiple concurrent orders decrement చేస్తే, race condition వల్ల actual stock కంటే ఎక్కువ orders accept అయ్యే ప్రమాదం ఉంది — ఇది real, costly production bug pattern (overselling). English: In e-commerce, multiple concurrent orders decrementing "available stock count" without proper synchronization is a real, costly production bug pattern — a race condition here literally causes overselling (accepting more orders than actual inventory), directly costing money and customer trust.

Interview Answer

"A race condition happens when multiple threads access shared mutable state concurrently and the outcome depends on unpredictable timing. `count++` is the classic example — it's not atomic; it's read-increment-write, and if two threads interleave those steps, one increment gets silently lost, producing a final count lower than the true number of increments performed."

Deep Interview Answer

"The root cause is that source-code-level 'simple' operations don't map to single, indivisible CPU/bytecode instructions — atomicity must be explicitly established through language mechanisms: `synchronized` (mutual exclusion, Ch.4), atomic classes using CAS (Ch.7), or explicit

locks (Ch.8). Crucially, a race condition bug is inherently non-deterministic and timing-dependent — it may not manifest in testing (especially on a lightly-loaded machine or low thread counts) yet appear reliably under real production load, which is exactly why concurrency bugs are notoriously hard to catch before production and require deliberate, principled design rather than empirical 'it worked when I tested it' confidence."

Cross Questions

- Q: Why is `count++` not atomic even though it looks like one operation? → A: It's actually a read-modify-write sequence at the bytecode level — three separate steps that can be interleaved by another thread between any of them.
- Q: Does reducing the number of threads eliminate a race condition? → A: No — it only reduces the *probability* of the interleaving that triggers the bug; the underlying non-atomicity is still present and can still manifest.
- Q: Is a race condition always about lost updates? → A: Lost updates are the classic example, but race conditions broadly cover any scenario where concurrent access to shared mutable state produces a result dependent on unpredictable interleaving — including reading partially-updated composite state.

Tricky Questions

- Q: If `counter` were declared `volatile` (Ch.5), would that fix this race condition? → A: No — `volatile` only guarantees visibility of writes across threads, not atomicity of compound read-modify-write operations like `++`; this is a very common and important misconception to correct.
- Q: Can a race condition occur on purely local (stack) variables? → A: No — local variables are private to each thread's own stack frame (Book 03, Ch.4); race conditions require *shared* state, which in Java means heap-allocated objects (instance/static fields) accessible from multiple threads.

Coding Exercise

L1: Reproduce the race condition demo and run it multiple times, observing different (wrong) final counts each time. **L2:** Fix it using `synchronized` (preview, full detail in Ch.4) and verify the count is now always correct. **L3:** Deliberately mark `counter` as `volatile` and confirm the race condition still occurs (proving `volatile` doesn't fix atomicity). **L4 (Interview):** Explain, step by step, exactly how two threads can lose an update to `count++`. **L5 (Senior):** Diagnose a reported "occasional incorrect inventory count" production bug, given the symptom only appears under high load — connect it to race conditions and propose a fix. **L6 (Mastery):** Explain, from memory, why race condition bugs are non-deterministic and can pass all tests yet fail in production.

CHAPTER 4 — `synchronized`, Deep Dive

Telugu Explanation

`synchronized` keyword ఒక **intrinsic lock (monitor)** వాడి, ఒకేసారి **ఒక్క thread మాత్రమే** ఆ locked code block/method ని execute చేయగలిగేలా చేస్తుంది — ఇతర threads block అయ్యి wait చేస్తాయి. Java లో ప్రతి object కి తనదైన monitor ఉంటుంది. Two forms: **synchronized method** (whole method, lock = `this` instance నా లేదా `static` methods కి Class object నా) మరియు **synchronized block** (finer-grained, explicit lock object తో).

Professional English Explanation

`synchronized` uses an **intrinsic lock (monitor)** to ensure only **one thread at a time** can execute the locked code block/method — other threads attempting to enter block, waiting. Every Java object has its own monitor. Two forms: **synchronized method** (locks the whole method, using `this` for instance methods or the `Class` object for `static` methods) and **synchronized block** (finer-grained, with an explicit lock object).

Java Code

```
public class SynchronizedDemo {
    private int counter = 0;
    private final Object lock = new Object();

    public synchronized void incrementMethodLevel() {           // lock = 'this'
        counter++;
    }

    public void incrementBlockLevel() {
        synchronized (lock) {                                   // lock = explicit
private object (finer control)
            counter++;
        }
    }

    public static synchronized void staticSyncMethod() {        // lock =
SynchronizedDemo.class
        System.out.println("Static synchronized method running on: " +
Thread.currentThread().getName());
    }

    public int getCounter() { return counter; }

    public static void main(String[] args) throws InterruptedException {
        SynchronizedDemo demo = new SynchronizedDemo();
        int numThreads = 10, incrementsPerThread = 100_000;
        Thread[] threads = new Thread[numThreads];

        for (int i = 0; i < numThreads; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < incrementsPerThread; j++) demo.incrementBlockLevel();
            });
        }
        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join();

        System.out.println("Expected: " + (numThreads * incrementsPerThread));
        System.out.println("Actual (with synchronized, always correct): " + demo.getCounter());

        // wait()/notify() - inter-thread communication using the SAME monitor as synchronized
        Object monitor = new Object();
        Thread waiter = new Thread(() -> {
            synchronized (monitor) {
                System.out.println("Waiter: waiting...");
                try { monitor.wait(); } catch (InterruptedException ignored) {}    // releases
the lock while waiting!
                System.out.println("Waiter: notified, resuming!");
            }
        });
        waiter.start();
        Thread.sleep(100);
        synchronized (monitor) {
            System.out.println("Notifier: sending notify()");
            monitor.notify();                                                    // wakes
the waiting thread
        }
        waiter.join();
    }
}
```

```
}  
}
```

Output (illustrative)

```
Expected: 1000000  
Actual (with synchronized, always correct): 1000000  
Waiter: waiting...  
Notifier: sending notify()  
Waiter: notified, resuming!
```

Internal Working

- Every Java object has an associated **monitor** (intrinsic lock); `synchronized` acquires that monitor before entering the block and releases it automatically on exit — even if an exception propagates out (similar in spirit to `finally`'s guarantee, Book 04, Ch.2), which is a genuine safety advantage over manually acquired locks (Ch.8) that must be released in a `finally` block explicitly.
- Using a **dedicated private lock object** (`private final Object lock`) rather than `this` for block synchronization is a best practice — locking on `this` exposes the lock to external code (anyone with a reference to your object could `synchronized(yourObject)` too, potentially causing unexpected contention or even deadlock, Ch.9), whereas a private lock object is fully encapsulated.
- `wait()` / `notify()` / `notifyAll()` **must** be called while holding the monitor for that object (inside a `synchronized` block on it), and `wait()` **does** release the lock while the thread is waiting (unlike `sleep()`, Ch.1's tricky question) — allowing another thread to acquire the same lock to eventually call `notify()`.
- `synchronized` is **reentrant** — a thread already holding a lock can re-acquire the same lock (e.g., calling another `synchronized` method on the same object from within a `synchronized` method) without deadlocking itself.

Real-World Example

Telugu: Bank account `withdraw()` / `deposit()` methods `synchronized` చేయకపోతే, రెండు concurrent transactions ఒకే account balance ని corrupt చేయచ్చు — ఇది Book 02, Ch.2 లో మనం చూసిన encapsulation example కి, ఇప్పుడు concurrency safety layer add చేయడం. English: Bank account `withdraw()` / `deposit()` methods (Book 02, Ch.2's encapsulation example) genuinely need `synchronized` (or a lock, Ch.8) in a concurrent environment — without it, two simultaneous transactions can corrupt the shared balance exactly like this chapter's race condition, this time with real financial consequences.

Interview Answer

"`synchronized` uses an object's intrinsic monitor to ensure only one thread executes the locked code at a time, automatically releasing the lock on exit (even via exception). Method-level synchronization locks on `this` (or the Class object for static methods); block-level synchronization with a dedicated private lock object is preferred for finer control and encapsulation."

`wait()` / `notify()` / `notifyAll()` must be called while holding that same monitor, and `wait()` uniquely releases the lock while waiting."

Deep Interview Answer

"Locking on `this` for block synchronization is a subtle but real anti-pattern — since `this` is publicly accessible, any external code holding a reference to your object can synchronize on it too, creating hidden coupling and contention (or deadlock risk, Ch.9) that your class has no control over. A private, dedicated lock object fully encapsulates the locking strategy as an implementation detail invisible to callers — this is directly analogous to Book 02's encapsulation principle, now applied specifically to concurrency control."

Cross Questions

- Q: Is `synchronized` released if an exception is thrown inside the block? → A: Yes — the JVM guarantees monitor release on any exit path from a `synchronized` block/method, exception or normal completion, just like `finally` (Book 04, Ch.2).
- Q: What does it mean that `synchronized` is reentrant? → A: A thread already holding a lock can acquire it again (e.g., a `synchronized` method calling another `synchronized` method on the same object) without blocking itself — the JVM tracks a per-thread hold count.
- Q: Why must `wait()` / `notify()` be called from within a `synchronized` block on the same object? → A: `IllegalMonitorStateException` is thrown otherwise — these methods coordinate access to the monitor itself, which only makes sense if the calling thread currently owns that monitor.

Tricky Questions

- Q: If Thread A calls `wait()` inside a `synchronized(lock)` block, does it still hold `lock` while waiting? → A: No — `wait()` specifically releases the lock while the thread is suspended, which is essential; otherwise no other thread could ever acquire the lock to call `notify()`, causing permanent deadlock.
- Q: Does locking on `this` in an instance method and locking on `ClassName.class` in a static method provide mutual exclusion between the two? → A: No — they use entirely different monitors (an instance's monitor vs the `Class` object's monitor), so a thread in the static `synchronized` method and another thread in the instance `synchronized` method can run concurrently without any blocking between them.

Coding Exercise

L1: Fix the Ch.3 race condition demo using `synchronized` and verify the count is always correct across multiple runs. **L2:** Refactor a method locking on `this` to instead lock on a dedicated private `Object`, explaining the encapsulation benefit. **L3:** Build a simple producer-consumer pair using `wait()` / `notify()` directly (a manual preview of Part 2's `BlockingQueue`). **L4 (Interview):** Explain why locking on `this` is discouraged compared to a private lock object. **L5 (Senior):** Review a `BankAccount` class's `withdraw()` / `deposit()` methods for thread safety, adding proper synchronization and justifying the lock granularity chosen. **L6 (Mastery):** Explain, from memory,

why `wait()` releases the lock but `sleep()` does not, and why that distinction is essential for correct inter-thread communication.

CHAPTER 5 — `volatile` & Visibility

Telugu Explanation

Multi-core CPUs లో, ప్రతి core తనదైన **cache** కలిగి ఉంటుంది — ఒక thread ఒక variable ని update చేస్తే, ఆ update వెంటనే మిగతా threads కి (వేరే core meీద running) కనిపించకపోవచ్చు, అది **main memory** కి flush అయ్యేవరకు. `volatile` keyword ఈ **visibility** సమస్యని పరిష్కరిస్తుంది — ప్రతి read direct గా main memory నుండి, ప్రతి write direct గా main memory కి జరిగేలా guarantee చేస్తుంది. కానీ **`volatile` atomicity ఇవ్వదు** (Ch.3 లో చూసినట్టు).

Professional English Explanation

On multi-core CPUs, each core has its own cache — a write by one thread may not be immediately visible to other threads (running on different cores) until it's flushed to main memory. `volatile` solves this **visibility** problem — guaranteeing every read comes directly from main memory and every write goes directly to main memory (bypassing CPU-local caching for that variable), and additionally guarantees ordering (Ch.6) around accesses to that variable. But **`volatile` does NOT provide atomicity** for compound operations (as shown in Ch.3).

Java Code

```
public class VolatileDemo {
    static volatile boolean running = true;        // visibility-critical flag
    static int nonVolatileCounter = 0;             // for contrast

    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            int localCount = 0;
            while (running) {                      // reads 'running' fresh from main
memory each time
                localCount++;
            }
            System.out.println("Worker stopped after observing running=false. Local iterations:
" + localCount);
        });

        worker.start();
        Thread.sleep(100);
        System.out.println("Main: setting running = false");
        running = false;                          // write is IMMEDIATELY visible to
worker thread
        worker.join();
        System.out.println("Main: worker has terminated cleanly");

        // Demonstrating volatile does NOT fix atomicity (revisit of Ch.3's lesson)
        VolatileCounterDemo.run();
    }
}

class VolatileCounterDemo {
    static volatile int counter = 0;               // visibility fixed, but NOT atomicity

    static void run() throws InterruptedException {
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(() -> { for (int j = 0; j < 100_000; j++) counter++; });
        }
        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join();
        System.out.println("volatile counter (still likely WRONG, proves volatile != atomic): "
+ counter);
    }
}
```

Output (illustrative)

```
Main: setting running = false
Worker stopped after observing running=false. Local iterations: 48213902
Main: worker has terminated cleanly
volatile counter (still likely WRONG, proves volatile != atomic): 974512
```

Internal Working

- Without `volatile`, the JIT compiler (Book 03, Ch.3) is legally permitted to **cache** the `running` flag's value in a CPU register for the loop's duration (a valid optimization for a value it doesn't

know is being mutated by another thread), potentially making the worker loop **never observe** `running` becoming `false` at all — a real, documented failure mode, not just a theoretical concern.

- `volatile` prevents this specific optimization for that variable — every access genuinely goes to main memory (or at least, is guaranteed visible across the memory hierarchy via the JMM, Ch.6) — but it does nothing about the read-modify-write non-atomicity from Ch.3, since `counter++` is still three separate operations, each individually visible/ordered correctly, but not indivisible as a whole.
- `volatile` writes also establish a **happens-before** relationship (Ch.6) — a `volatile` write by one thread is guaranteed visible, along with everything that thread did *before* that write, to any thread that subsequently reads that same `volatile` variable — this is a stronger and more subtle guarantee than "just visibility of that one variable."

Real-World Example

Telugu: "shutdown flag" పద్ధతిలో worker threads ని gracefully stop చేయడానికి `volatile boolean running` వాడటం production code లో extremely common pattern — flag ని `volatile` గా declare చేయకపోతే, worker thread ఎప్పటికీ update ని చూడకపోవచ్చు, infinite loop లో ఉండిపోవచ్చు. English: The "volatile shutdown flag" pattern for gracefully stopping worker threads is an extremely common real production idiom — forgetting `volatile` here is a genuine, documented bug class where a worker thread can spin forever, having cached a stale `true` value and never observing the shutdown signal.

Interview Answer

"`volatile` guarantees visibility — every write is immediately visible to other threads, and every read gets the latest value, preventing the JIT from caching the variable in a way that hides updates from other threads. It also establishes a happens-before relationship for ordering. Critically, `volatile` does NOT provide atomicity for compound operations like `count++` — that still requires `synchronized`, atomic classes (Ch.7), or locks (Ch.8)."

Cross Questions

- Q: Does `volatile` make `count++` thread-safe? → A: No — this is the single most common `volatile` misconception; it fixes visibility, not the multi-step non-atomicity of increment.
- Q: What's the classic real-world use case for `volatile`? → A: A simple boolean flag (like a shutdown signal) that one thread sets and other threads only read, with no compound read-modify-write logic involved.
- Q: What additional guarantee does `volatile` provide beyond "the latest value is visible"? → A: A happens-before ordering guarantee (Ch.6) — everything the writing thread did before the `volatile` write is also guaranteed visible to the reading thread after it reads that value.

Tricky Questions

- Q: If two threads both do `volatile int x; x++;` concurrently, can updates still be lost? → A: Yes — exactly as demonstrated in this chapter's demo; `volatile` only ensures each individual

read/write of `x` is visible and ordered, not that the read-then-write sequence is atomic as a whole.

- Q: Does making an object reference `volatile` make the object's own fields thread-safe? → A: No — `volatile` only affects the reference variable itself (ensuring visibility of *which object* it points to); the referenced object's internal mutable state still needs its own synchronization if accessed concurrently.

Coding Exercise

L1: Reproduce the classic "worker never sees the shutdown flag" bug by removing `volatile` and (if reproducible on your JVM/settings) observing the worker loop forever. **L2:** Confirm that a `volatile` counter still loses updates under concurrent increment, proving visibility $\neq$ atomicity. **L3:** Build a graceful-shutdown worker pattern using a `volatile boolean running` flag, correctly checked in the worker's loop condition. **L4 (Interview):** Explain precisely what `volatile` guarantees and what it does NOT guarantee. **L5 (Senior):** Review a codebase using `volatile` on a counter field expected to be thread-safe — identify the bug and fix it with an atomic class (preview of Ch.7) or proper synchronization. **L6 (Mastery):** Explain, from memory, why the JIT compiler might cache a non-volatile variable in a register, and why that specifically breaks a naive "flag-based" shutdown loop.

CHAPTER 6 — The Java Memory Model: Happens-Before

Telugu Explanation

Java Memory Model (JMM) అనేది threads ఎలా shared memory ని access చేస్తాయో, ఏ ordering/visibility guarantees ఉంటాయో define చేసే **formal specification**. దీని core concept: **happens-before relationship** — action A "happens-before" action B అయితే, A యొక్క effects (writes) B కి guaranteed గా visible అవుతాయి. Happens-before ఏర్పడే ప్రధాన మార్గాలు: program order (ఒకే thread లో), synchronized block exit → తర్వాత entry, `volatile` write → తర్వాత read, `Thread.start()` → thread body, thread body → `Thread.join()` return.

Professional English Explanation

The Java Memory Model (JMM) is the formal specification defining how threads interact through shared memory — what ordering and visibility guarantees actually exist. Its core concept is the **happens-before relationship**: if action A happens-before action B, then A's effects (writes) are guaranteed visible to B. Key happens-before edges: program order (within a single thread), a `synchronized` block's exit happens-before a subsequent thread's entry to a block on the same lock, a `volatile` write happens-before a subsequent read of that same variable, `Thread.start()` happens-before anything in the started thread's body, and everything in a thread's body happens-before a `Thread.join()` returning in the joining thread.

Diagram — Happens-Before Edges

```
THREAD A                                THREAD B
-----                                -----
x = 10;                                \
y = 20;                                } program order
synchronized(lock) {                  }
    sharedState = 100; } ---- happens-before ----> synchronized(lock) {
}                                     / (lock release -> acquire)   read sharedState; // guaranteed to see
100                                     }

flag = true; (volatile write) ----- happens-before -----> if (flag) { ... } (volatile read)
written                                                         // guaranteed to see everything
                                                                // before the volatile write

too!

Thread t = new Thread(...);
t.start(); ----- happens-before -----> [everything inside t's run()]

[everything inside t's run()] ----- happens-before -----> t.join(); (in the joining thread,
AFTER join() returns)
```

Java Code

```
public class HappensBeforeDemo {
    static int data = 0;           // NOT volatile - protected instead by the happens-
    before edges below
    static volatile boolean ready = false;

    public static void main(String[] args) throws InterruptedException {
        Thread writer = new Thread(() -> {
            data = 42;              // (1) regular write
            ready = true;           // (2) volatile write - happens-before any
            subsequent read of 'ready'
        });

        Thread reader = new Thread(() -> {
            while (!ready) { /* spin-wait for the volatile flag */ } // (3) volatile read
            System.out.println("data = " + data);                    // (4) GUARANTEED to
            see 42, not 0!
        });

        reader.start();
        writer.start();
        writer.join();
        reader.join();

        // Thread.start()/join() happens-before demonstration
        int[] resultHolder = new int[1];
        Thread computeThread = new Thread(() -> resultHolder[0] = 99); // no
        synchronization needed here
        computeThread.start();
        computeThread.join();                                           // happens-before
        edge guarantees visibility
        System.out.println("Result after join(): " + resultHolder[0]); // GUARANTEED to
        see 99
    }
}
```

Output

```
data = 42
Result after join(): 99
```

Internal Working

- The reason `data` (a plain, non-`volatile` `int`) is **guaranteed** to be seen correctly as `42` by the reader thread is entirely due to the happens-before chain: `data = 42` happens-before `ready = true` (program order within the writer thread), and `ready = true` (a volatile write) happens-before the reader's `while(!ready)` loop observing `true` (volatile write-then-read edge) — by transitivity, `data = 42` happens-before the reader's subsequent read of `data`. This is the JMM's most important practical pattern: **a single volatile "ready" flag can safely "publish" an entire batch of prior writes**, without needing every individual field to be `volatile`.
- Similarly, `Thread.start()` / `Thread.join()` form happens-before edges specifically so that a thread's entire body of work is guaranteed visible to whoever successfully joins it — this is why

the `resultHolder[0] = 99` write, despite being on a plain (non-volatile, non-synchronized) array, is safely visible after `join()` returns.

- Without any happens-before relationship established between two threads' accesses to the same variable, the JMM makes **no guarantee whatsoever** about visibility or ordering — the compiler, JIT, and even the CPU itself are all permitted to reorder, cache, or delay operations in ways that only matter for genuinely concurrent, unsynchronized access (single-threaded correctness is always preserved).

Real-World Example

Telugu: "Double-checked locking" singleton pattern (Book 18) `volatile` వాడకపోతే broken అవుతుంది సరిగ్గా ఈ JMM reordering కారణంగా — object partially-constructed state ఇతర threads కి visible అయ్యే ప్రమాదం ఉంటుంది. `ready` flag pattern ఇక్కడ చూపించిన idiom ఖచ్చితంగా production code (config loading, cache warm-up) లో వాడతారు. English: The famous "double-checked locking" singleton pattern (Book 18) is broken without `volatile` precisely because of JMM reordering — another thread can observe a partially-constructed object through a non-volatile reference. The `volatile ready` flag pattern shown here is exactly the idiom used in production for config loading and cache warm-up — safely publishing a whole batch of setup writes via one flag.

Interview Answer

"The Java Memory Model defines happens-before relationships that guarantee visibility and ordering between threads. Key edges: program order within a thread, a synchronized block's unlock happens-before a subsequent lock on the same monitor, a volatile write happens-before a subsequent read of that variable, and `Thread.start()/join()` bracket a thread's entire body. Without an established happens-before relationship, the JMM makes no visibility/ordering guarantee at all between threads' accesses to shared state."

Deep Interview Answer

"The most powerful practical consequence of happens-before is transitivity: you don't need every shared field to be individually `volatile` or synchronized — you only need ONE properly-published happens-before edge (a single `volatile` flag, or entering/exiting the same lock) to safely carry an arbitrary number of prior plain writes across to another thread. This is exactly how safe publication works for immutable objects (Ch.10) and the 'ready flag' pattern — understanding this is what separates memorizing 'volatile makes things visible' from genuinely understanding why entire object graphs can be safely shared with minimal synchronization overhead."

Cross Questions

- Q: If Thread A writes a non-volatile field and Thread B reads it with no happens-before relationship established between them, what does the JMM guarantee? → A: Nothing — Thread B might see the old value, the new value, or (for non-atomic 64-bit types without volatile, a rare edge case) even a torn/partial value; this is genuinely undefined behavior from the JMM's perspective.
- Q: Does `synchronized` only provide mutual exclusion, or does it also establish happens-before? → A: Both — mutual exclusion (Ch.4) AND a happens-before edge between the unlocking thread

and the next thread to successfully lock the same monitor.

- Q: Why does the "volatile ready flag" pattern only need ONE field to be volatile? → A: Because happens-before is transitive — the volatile write/read pair creates a single strong edge that carries every earlier, plain write in the writer thread across to every later read in the reader thread, once the flag is observed as set.

Tricky Questions

- Q: If Thread A does `x = 1; y = 2;` (both non-volatile, no synchronization) and Thread B does `System.out.println(y); System.out.println(x);` with no happens-before relationship, can B print `y=2, x=0`? → A: Yes — without any happens-before edge, the JMM permits this (the compiler/CPU may reorder independent writes, and B may see them in any order or not at all yet), which is precisely why unsynchronized cross-thread access to plain fields is unsafe regardless of how "unlikely" a particular reordering seems.
- Q: Does calling `Thread.join()` on a thread that already finished still establish the happens-before edge? → A: Yes — `join()`'s happens-before guarantee applies regardless of whether the joining thread had to actually wait; it's about establishing the relationship, not about the timing of when it's observed.

Coding Exercise

L1: Reproduce the `data / ready` volatile-flag publishing pattern and verify the reader always sees `data = 42` correctly. **L2:** Remove `volatile` from `ready` and discuss (you may not always be able to reliably reproduce it, since JMM violations are legal-but-not-mandatory) why the guarantee is now gone even if it "happens to work" in your test. **L3:** Demonstrate the `Thread.start() / join()` happens-before edge safely publishing a plain array write, like the demo's `resultHolder` example. **L4 (Interview):** List the 4 key happens-before edges from memory. **L5 (Senior):** Explain why the double-checked locking singleton pattern requires a `volatile` field, referencing JMM reordering risk. **L6 (Mastery):** Explain, from memory, why happens-before's transitivity means only ONE volatile flag can safely "publish" an entire batch of prior plain writes.

CHAPTER 7 — Atomic Classes & Compare-And-Swap

Telugu Explanation

`java.util.concurrent.atomic` package (`AtomicInteger`, `AtomicLong`, `AtomicReference`, etc.) **lock-free, atomic** compound operations (increment, compare-and-set) అందిస్తుంది — `synchronized` కంటే తక్కువ overhead తో, ముఖ్యంగా high-contention scenarios లో better throughput ఇస్తుంది. దీని internal mechanism: **CAS (Compare-And-Swap)** — CPU-level atomic instruction, "current value ఇంకా నేను expect చేసిన దానికే సమానంగా ఉంటే, కొత్త value తో replace చేయి" అని atomically చేస్తుంది.

Professional English Explanation

The `java.util.concurrent.atomic` package (`AtomicInteger`, `AtomicLong`, `AtomicReference`, etc.) provides **lock-free**, genuinely atomic compound operations (increment, compare-and-set) — with lower overhead than `synchronized` in many scenarios, and better throughput under high contention. Internally, it's built on **CAS (Compare-And-Swap)** — a CPU-level atomic instruction: "if the current value still equals what I expect, atomically replace it with the new value; otherwise, fail" — with the caller typically retrying in a loop until it succeeds.

Java Code

```
import java.util.concurrent.atomic.*;

public class AtomicClassesDemo {
    static AtomicInteger atomicCounter = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        int numThreads = 10, incrementsPerThread = 100_000;
        Thread[] threads = new Thread[numThreads];

        for (int i = 0; i < numThreads; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < incrementsPerThread; j++) atomicCounter.incrementAndGet();
            });
        }
        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join();

        System.out.println("Expected: " + (numThreads * incrementsPerThread));
        System.out.println("Atomic result (always correct, no synchronized needed): " +
            atomicCounter.get());

        // compareAndSet - the primitive CAS operation directly
        AtomicInteger cas = new AtomicInteger(10);
        boolean success1 = cas.compareAndSet(10, 20);           // expected=10 matches
        // current=10 -> succeeds
        boolean success2 = cas.compareAndSet(10, 30);           // expected=10 does NOT
        // match current=20 -> fails
        System.out.println("First CAS success: " + success1 + ", value now: " + cas.get());
        System.out.println("Second CAS success: " + success2 + ", value still: " + cas.get());

        // AtomicReference - atomic updates to object references (e.g., for lock-free
        // algorithms)
        AtomicReference<String> atomicRef = new AtomicReference<>("initial");
        atomicRef.compareAndSet("initial", "updated");
        System.out.println("AtomicReference value: " + atomicRef.get());

        // updateAndGet with a lambda (Book 07) - atomic custom compound operation
        AtomicInteger customUpdate = new AtomicInteger(5);
        int newVal = customUpdate.updateAndGet(v -> v * v);      // atomically squares the
        // value
        System.out.println("Squared atomically: " + newVal);
    }
}
```

Output

```
Expected: 1000000
Atomic result (always correct, no synchronized needed): 1000000
First CAS success: true, value now: 20
Second CAS success: false, value still: 20
Third CAS success message omitted for brevity
AtomicReference value: updated
Squared atomically: 25
```

Internal Working — CAS Loop

```
compareAndSet(expected, newValue):
    atomically {
        if (currentValue == expected) {
            currentValue = newValue
            return true
        } else {
            return false    // someone else changed it first - caller must retry with the NEW
current value
        }
    }

incrementAndGet() is internally implemented roughly as:
do {
    current = get()
    next = current + 1
} while (!compareAndSet(current, next))    // retry loop until no other thread interfered
return next
```

- CAS is a **single, hardware-supported atomic CPU instruction** (e.g., x86's `CMPXCHG`) — this is why it can be genuinely atomic without needing a full lock/monitor (Ch.4) acquisition, avoiding the overhead of thread blocking/context-switching entirely in the common (uncontended) case.
- This is called **optimistic concurrency control**: instead of *preventing* other threads from touching the value (pessimistic locking, like `synchronized`), CAS *allows* concurrent attempts and simply retries if another thread "won the race" — under low-to-moderate contention, this is typically faster than lock-based approaches, since most attempts succeed on the first try with no blocking at all.
- Under very **high** contention (many threads CAS-looping on the same value simultaneously), performance can actually degrade compared to a lock, since many threads repeatedly fail and retry ("spinning") — this is a genuine, known trade-off, not a universal "atomics are always faster" rule.

Real-World Example

Telugu: Request counters, metrics collection (ఉదా. "total API calls served") — high-frequency, simple increment operations కి `AtomicLong` ఖచ్చితంగా సరిపోతుంది, `synchronized` overhead లేకుండా. English: Request counters and metrics collection (e.g., "total API calls served") — high-frequency, simple increment operations — are the classic real production `AtomicLong` / `AtomicInteger` use case, avoiding `synchronized` overhead entirely for exactly the kind of simple compound operation atomics are built for.

Interview Answer

"Atomic classes (`AtomicInteger`, `AtomicLong`, `AtomicReference`) provide lock-free, genuinely atomic compound operations built on Compare-And-Swap (CAS) — a hardware-level atomic instruction that conditionally updates a value only if it still matches an expected value, retrying otherwise. This is optimistic concurrency control, typically faster than `synchronized` under low-to-moderate contention since it avoids blocking/context-switching, though it can degrade under very high contention due to repeated retry spinning."

Cross Questions

- Q: What does `compareAndSet(expected, newValue)` return if the current value doesn't match `expected`? → A: `false`, and it leaves the value unchanged — the caller is responsible for retrying with updated logic if needed (as `incrementAndGet()` does internally).
- Q: Why is CAS called "optimistic" concurrency control? → A: It optimistically attempts the update assuming no conflict, only detecting and retrying after the fact if another thread interfered — as opposed to "pessimistic" locking (`synchronized`), which prevents any potential conflict upfront by blocking other threads entirely.
- Q: Is CAS always faster than `synchronized`? → A: No — under very high contention, CAS-based retry loops can spend significant CPU time repeatedly failing and retrying, sometimes underperforming a lock that simply queues waiting threads instead.

Tricky Questions

- Q: Can CAS suffer from the "ABA problem"? → A: Yes — if a value changes from A to B and back to A between a thread's read and its CAS attempt, the CAS succeeds (since the value matches "A" again) even though the value *did* change in between — this can be a subtle bug for algorithms that assume "unchanged" means "nothing happened," and is specifically addressed by `AtomicStampedReference` (which adds a version stamp) for algorithms where it matters.
- Q: Does `AtomicInteger.incrementAndGet()` ever "lose" an update under contention, unlike plain `count++`? → A: No — it's genuinely atomic via its internal CAS retry loop; every call is guaranteed to eventually succeed and contribute its increment, never silently losing one (contrast directly with Ch.3's race condition).

Coding Exercise

L1: Fix the Ch.3 race condition demo using `AtomicInteger` instead of `synchronized`, and verify correctness. **L2:** Use `compareAndSet()` directly to implement a simple optimistic "update if unchanged" pattern. **L3:** Use `AtomicReference<T>` to implement a lock-free "latest value wins" cache slot. **L4 (Interview):** Explain CAS and why it's called optimistic concurrency control. **L5 (Senior):** Compare `AtomicLong` vs `synchronized` for a high-throughput request counter under both low and very high contention, explaining the trade-off. **L6 (Mastery):** Explain, from memory, the ABA problem and how `AtomicStampedReference` addresses it.

CHAPTER 8 — Locks: ReentrantLock, ReadWriteLock, StampedLock

Telugu Explanation

`java.util.concurrent.locks` package `synchronized` కంటే ఎక్కువ flexibility ఇచ్చే explicit lock objects అందిస్తుంది: `ReentrantLock` (`synchronized` లాంటిదే కానీ **`tryLock()` (timeout తో), interruptible locking, fairness policy** వంటి extra features తో), `ReadWriteLock` (multiple readers ఏకకాలంలో access చేయగలరు, writer exclusive access కావాలంటే), `StampedLock` (Java 8+, optimistic reading support తో, ఇంకా better read-heavy performance).

Professional English Explanation

`java.util.concurrent.locks` provides explicit lock objects offering more flexibility than `synchronized`: `ReentrantLock` (functionally similar to `synchronized` but with extras — timed/interruptible `tryLock()`, a fairness policy option), `ReadWriteLock` (allows multiple concurrent readers, but exclusive access for a writer), and `StampedLock` (Java 8+, adding optimistic-read support for even better read-heavy performance).

Java Code

```
import java.util.concurrent.locks.*;
import java.util.concurrent.*;

public class LocksDemo {
    static final ReentrantLock reentrantLock = new ReentrantLock();
    static int sharedCounter = 0;

    static final ReadWriteLock rwLock = new ReentrantReadWriteLock();
    static int cachedValue = 100;

    public static void main(String[] args) throws InterruptedException {
        // ReentrantLock: MUST manually unlock, ideally in a finally block
        Runnable incrementTask = () -> {
            for (int i = 0; i < 100_000; i++) {
                reentrantLock.lock();
                try {
                    sharedCounter++;
                } finally {
                    reentrantLock.unlock(); // CRITICAL - always unlock in
finally
                }
            }
        };
        Thread t1 = new Thread(incrementTask), t2 = new Thread(incrementTask);
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("ReentrantLock counter (expected 200000): " + sharedCounter);

        // tryLock with timeout - avoids indefinite blocking
        boolean acquired = reentrantLock.tryLock(50, TimeUnit.MILLISECONDS);
        System.out.println("tryLock acquired immediately (lock is free): " + acquired);
        if (acquired) reentrantLock.unlock();

        // ReadWriteLock: many readers OR one exclusive writer, never both simultaneously
        Runnable reader = () -> {
            rwLock.readLock().lock();
            try {
                System.out.println(Thread.currentThread().getName() + " read cachedValue=" +
cachedValue);
                Thread.sleep(10);
            } catch (InterruptedException ignored) {}
            finally { rwLock.readLock().unlock(); }
        };
        Runnable writer = () -> {
            rwLock.writeLock().lock();
            try {
                cachedValue = 200;
                System.out.println(Thread.currentThread().getName() + " wrote cachedValue=200
(exclusive access)");
            } finally { rwLock.writeLock().unlock(); }
        };

        Thread r1 = new Thread(reader, "reader-1"), r2 = new Thread(reader, "reader-2"), w1 =
new Thread(writer, "writer-1");
        r1.start(); r2.start();
        r1.join(); r2.join();
        w1.start(); w1.join();
    }
}
```

```
}
}
```

Output (illustrative)

```
ReentrantLock counter (expected 200000): 200000
tryLock acquired immediately (lock is free): true
reader-1 read cachedValue=100
reader-2 read cachedValue=100
writer-1 wrote cachedValue=200 (exclusive access)
```

Internal Working & Comparison

Feature	synchronized	ReentrantLock	ReadWriteLock	StampedLock
Automatic release	Yes (even on exception)	No — must call <code>unlock()</code> manually, ideally in <code>finally</code>	Same as ReentrantLock per read/write lock	Same, plus optimistic mode needs manual validation
Timed/interruptible acquire	No	Yes (<code>tryLock(timeout)</code> , <code>lockInterruptibly()</code>)	Yes, per read/write lock	Yes
Fairness policy option	No	Yes (<code>new ReentrantLock(true)</code>)	Yes	No
Multiple concurrent readers	No	No	Yes (read lock is shared)	Yes (optimistic reads are lock-free)
Condition variables (multiple)	Only one implicit (<code>wait / notify</code>)	Multiple (<code>newCondition()</code>)	Per read/write lock	No

- `ReentrantLock` **requires manual `unlock()`** , almost always in a `finally` block — forgetting this (e.g., an early `return` before `unlock()`) is a genuine, serious bug that permanently deadlocks any future contender for that lock; this is the single biggest practical risk of explicit locks versus `synchronized` 's automatic release.
- `ReadWriteLock` dramatically improves throughput for **read-heavy** workloads (e.g., a rarely-updated cache) — many threads can hold the read lock simultaneously, only blocking when a writer needs exclusive access, unlike `synchronized` / `ReentrantLock` , which serialize even read-only access.
- `StampedLock` 's **optimistic read** mode (`tryOptimisticRead()`) doesn't block at all — it reads without acquiring any lock, then validates afterward (`validate(stamp)`) whether a write occurred in the meantime, retrying with a full read lock only if validation fails — offering even better read throughput than `ReadWriteLock` for very read-heavy, low-write scenarios, at the cost of a more complex, error-prone usage pattern.

Real-World Example

Telugu: Configuration cache — frequently read, rarely updated — `ReadWriteLock / StampedLock` కి ideal use case; ఎన్ని threads అయినా ఏకకాలంలో config read చేయగలరు, update జరిగినప్పుడు మాత్రమే brief exclusive lock అవసరం. English: A configuration cache — read very frequently, updated rarely — is the textbook `ReadWriteLock / StampedLock` use case: any number of threads can read concurrently, with only brief exclusive locking during the rare update, dramatically outperforming a plain `synchronized / ReentrantLock` approach that would serialize every single read.

Interview Answer

"`java.util.concurrent.locks` offers explicit locks beyond `synchronized`: `ReentrantLock` adds timed/interruptible acquisition and fairness policies but requires manual `unlock()` (critical to do in a `finally` block). `ReadWriteLock` allows multiple concurrent readers with exclusive writer access, ideal for read-heavy workloads. `StampedLock` (Java 8+) adds optimistic, lock-free reading for even better read-heavy throughput, at the cost of more complex correct usage."

Cross Questions

- Q: What's the biggest practical risk of `ReentrantLock` compared to `synchronized`? → A: Forgetting to call `unlock()` (especially on an exceptional or early-return code path) permanently blocks other threads waiting for that lock — always pair `lock()` with a `try { ... } finally { unlock(); }`.
- Q: Why would you choose `ReadWriteLock` over plain `ReentrantLock`? → A: When reads vastly outnumber writes — allowing concurrent reads dramatically improves throughput compared to serializing all access.
- Q: What does `StampedLock`'s optimistic read mode actually validate? → A: Whether any write occurred between the optimistic read attempt and the validation call — if a write happened, the optimistic read's result may be inconsistent and must be discarded, falling back to a full (blocking) read lock.

Tricky Questions

- Q: Is `ReentrantLock` reentrant in the same sense as `synchronized`? → A: Yes — a thread already holding a `ReentrantLock` can acquire it again (tracked by an internal hold count), exactly like `synchronized`'s reentrancy (Ch.4); the name literally describes this property.
- Q: Can a `ReadWriteLock`'s write lock be acquired while read locks are held by other threads? → A: No — the writer must wait until all current readers release their read locks; this is what guarantees exclusive access for writes, at the cost of potential writer starvation under continuous read load (a real, documented trade-off, connecting to Ch.9).

Coding Exercise

L1: Fix the Ch.3 race condition using `ReentrantLock`, being careful to unlock in a `finally` block. **L2:** Reproduce (in comments, if unsafe to actually run) what would happen if `unlock()` were accidentally skipped on an exception path. **L3:** Build a simple read-heavy cache using `ReadWriteLock`, demonstrating multiple concurrent readers via timestamps/logging. **L4**

(Interview): Explain why `ReentrantLock` requires manual unlocking and why that's a real risk compared to `synchronized`. **L5 (Senior):** Design a configuration cache using `StampedLock`'s optimistic read mode, explaining the validate-and-retry pattern. **L6 (Mastery):** Recreate the full comparison table (`synchronized` vs `ReentrantLock` vs `ReadWriteLock` vs `StampedLock`) from memory.

CHAPTER 9 — Deadlock, Livelock & Starvation

Telugu Explanation

Deadlock: రెండు (లేదా ఎక్కువ) threads ఒకదాని resource కోసం మరొకటి wait చేస్తూ, శాశ్వతంగా ఆగిపోవడం (circular wait). **Livelock:** threads deadlock కాకుండా active గానే ఉంటాయి, కానీ ఒకదానికొకటి constantly react చేస్తూ, ఏ progress చేయలేకపోవడం. **Starvation:** ఒక thread కి resource ఎప్పటికీ దొరకకపోవడం, ఇతర threads ఎప్పుడూ ముందు వెళ్తూ ఉండటం వల్ల.

Professional English Explanation

Deadlock: two or more threads each wait forever for a resource the other holds (circular wait), permanently frozen. **Livelock:** threads remain active (not blocked) but keep reacting to each other in a way that prevents any actual progress. **Starvation:** a thread never gets the resource/CPU time it needs because other threads are perpetually favored ahead of it.

Java Code — Deadlock (and how to avoid it)

```
public class DeadlockDemo {
    static final Object lockA = new Object();
    static final Object lockB = new Object();

    static void deadlockProneMethod1() {
        synchronized (lockA) {
            System.out.println("Thread1: holding lockA, waiting for lockB");
            try { Thread.sleep(50); } catch (InterruptedException ignored) {}
            synchronized (lockB) { // DEADLOCK RISK:
acquires B while holding A
                System.out.println("Thread1: acquired both locks");
            }
        }
    }

    static void deadlockProneMethod2() {
        synchronized (lockB) {
            System.out.println("Thread2: holding lockB, waiting for lockA");
            try { Thread.sleep(50); } catch (InterruptedException ignored) {}
            synchronized (lockA) { // DEADLOCK RISK:
acquires A while holding B - OPPOSITE ORDER!
                System.out.println("Thread2: acquired both locks");
            }
        }
    }

    // FIX: always acquire locks in a CONSISTENT GLOBAL ORDER to eliminate circular wait
    static void safeMethod1() {
        synchronized (lockA) { synchronized (lockB) { System.out.println("safeMethod1: got
both, consistent order"); } }
    }
    static void safeMethod2() {
        synchronized (lockA) { synchronized (lockB) { System.out.println("safeMethod2: got
both, consistent order"); } }
    }

    public static void main(String[] args) throws InterruptedException {
        // Demonstrating the FIX (running the actual deadlock-prone version would hang forever
- not run here)
        Thread t1 = new Thread(DeadlockDemo::safeMethod1);
        Thread t2 = new Thread(DeadlockDemo::safeMethod2);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println("No deadlock - both threads completed because lock order was
consistent.");
    }
}
```

Output

```
safeMethod1: got both, consistent order
safeMethod2: got both, consistent order
No deadlock - both threads completed because lock order was consistent.
```

Diagram — Deadlock's Circular Wait

```
Thread 1: holds lockA, wants lockB ---waiting for---> Thread 2
    ^
    |
    +----- waiting for ----- Thread 2: holds lockB, wants lockA
    |
    v
    +----- waiting for ----- Thread 1: holds lockA, wants lockB

Result: NEITHER thread can ever proceed. Permanent freeze.
```

The 4 Necessary Conditions for Deadlock (Coffman Conditions)

1. **Mutual exclusion** — a resource can only be held by one thread at a time.
2. **Hold and wait** — a thread holds one resource while waiting for another.
3. **No preemption** — a resource can't be forcibly taken from a thread holding it.
4. **Circular wait** — a cycle of threads, each waiting for a resource held by the next.

Breaking **any one** of these prevents deadlock — in practice, breaking **circular wait** (via consistent lock ordering, as shown in the fix above) is the most common and practical solution.

Real-World Example

Telugu: Bank transfer (`transferMoney(accountA, accountB, amount)`) — `accountA.lock()` తర్వాత `accountB.lock()` ఒక thread లో, `accountB.lock()` తర్వాత `accountA.lock()` మరో thread లో (reverse transfer) జరిగితే — ఇది classic real-world deadlock scenario. Fix: account IDs మీద ఆధారపడి, ఎప్పుడూ **చిన్న ID ముందు** lock చేయడం. English: A money-transfer method locking `accountA` then `accountB` in one call, while a concurrent reverse-transfer locks `accountB` then `accountA`, is the classic real-world deadlock scenario in banking systems — the standard fix is establishing a consistent lock order based on a stable property (e.g., always lock the account with the smaller ID first), eliminating the circular wait regardless of transfer direction.

Interview Answer

"Deadlock is a circular wait — each thread holds a resource another needs, and none can proceed, forever. Livelock is threads staying active but making no real progress, endlessly reacting to each other. Starvation is a thread perpetually denied a resource because others are favored. Deadlock's classic fix is enforcing a consistent global lock-acquisition order across all code paths, breaking the circular-wait condition — one of the four necessary Coffman conditions for deadlock."

Deep Interview Answer

"All four Coffman conditions (mutual exclusion, hold-and-wait, no preemption, circular wait) must hold simultaneously for deadlock to occur — breaking any single one prevents it. In practice, mutual exclusion and no-preemption are usually inherent to the resource type (you can't easily make a lock non-exclusive or preemptible), so real-world fixes almost always target hold-and-wait (acquire all needed locks atomically upfront, or use `tryLock()` with a timeout and back off/retry if not all can be acquired) or circular wait (a consistent global lock ordering). The 'consistent ordering' fix is preferred in practice because it requires no runtime retry logic and is provably deadlock-free by construction."

Cross Questions

- Q: What are the 4 Coffman conditions for deadlock? → A: Mutual exclusion, hold-and-wait, no preemption, circular wait — all four must hold simultaneously.
- Q: What's the most common practical fix for deadlock? → A: Enforcing a consistent global order in which locks are always acquired, eliminating circular wait.
- Q: How does livelock differ from deadlock? → A: In deadlock, threads are blocked and inactive forever; in livelock, threads remain active and keep executing, but their mutual reactions to each other prevent any actual forward progress (e.g., two threads repeatedly "politely" backing off for each other, forever).

Tricky Questions

- Q: Can `tryLock()` (Ch.8) with a timeout help avoid deadlock, and how? → A: Yes — instead of blocking indefinitely for a second lock (which risks hold-and-wait deadlock), a thread can attempt `tryLock(timeout)`, and if it fails, release any locks it currently holds and retry later — breaking the "wait forever" aspect of hold-and-wait.
- Q: Is starvation always caused by malicious or buggy code? → A: No — it can arise innocently from a naive/unfair scheduling or locking policy (e.g., a non-fair `ReentrantLock`, Ch.8, might statistically favor threads that happen to request the lock at opportune times) without any single piece of code being "wrong" in isolation.

Coding Exercise

L1: Write (but don't run indefinitely — add a timeout or run in a controlled test) the classic two-lock deadlock scenario, and observe it hang. **L2:** Fix it using consistent lock ordering, and confirm both threads now complete. **L3:** Fix the same scenario differently using `tryLock()` with a timeout and a retry/backoff loop (Ch.8). **L4 (Interview):** List the 4 Coffman conditions for deadlock from memory. **L5 (Senior):** Design a bank transfer method between two accounts that is provably deadlock-free regardless of transfer direction, using a consistent lock-ordering strategy based on account ID. **L6 (Mastery):** Explain, from memory, the difference between deadlock, livelock, and starvation, with a distinct example scenario for each.

CHAPTER 10 — Immutability & Thread-Safe Design + Mini Project

Telugu Explanation

Concurrency bugs నివారించడానికి అత్యంత powerful, simple technique: **shared mutable state ని eliminate చేయడం**. Immutable objects (Book 01, Ch.15; Book 02, Ch.15) — construction తర్వాత state మారదు కాబట్టి — **ఎలాంటి synchronization అవసరం లేకుండానే** thread-safe గా ఉంటాయి. ఈ chapter లో thread-safe class design principles మరియు Part 1 మొత్తం combine చేసే mini project చూద్దాం.

Professional English Explanation

The single most powerful, simplest technique for avoiding concurrency bugs: **eliminate shared mutable state entirely**. Immutable objects (Book 01, Ch.15; Book 02, Ch.15) are thread-safe **with zero synchronization needed**, since their state never changes after construction — there's nothing for concurrent access to race on. This chapter covers thread-safe class design principles and a Part 1 capstone mini project.

Java Code — Designing a Thread-Safe Immutable Class

```
import java.util.*;

public final class ImmutableThreadSafeDemo {

    static final class ImmutableOrder {                                // final class - can't be subtly
broken by subclassing
        private final String id;
        private final List<String> items;                            // mutable type internally -
needs defensive handling!
        private final double total;

        ImmutableOrder(String id, List<String> items, double total) {
            this.id = id;
            this.items = List.copyOf(items);                        // DEFENSIVE COPY (Book
05, Ch.12) - critical!
            this.total = total;
        }

        String id() { return id; }
        List<String> items() { return items; }                      // safe - already
immutable (List.copyOf)
        double total() { return total; }

        ImmutableOrder withAdditionalTotal(double extra) {          // "modification"
returns a NEW instance
            return new ImmutableOrder(id, items, total + extra);
        }
    }

    public static void main(String[] args) throws InterruptedException {
        List<String> mutableInput = new ArrayList<>(List.of("Laptop", "Mouse"));
        ImmutableOrder order = new ImmutableOrder("ORD1", mutableInput, 55000.0);

        mutableInput.add("Keyboard");                                // mutate the
ORIGINAL list after construction
        System.out.println("Order items unaffected by later external mutation: " +
order.items()); // still 2 items!

        ImmutableOrder updated = order.withAdditionalTotal(500.0);    // "modification"
= new object
        System.out.println("Original total unchanged: " + order.total());
        System.out.println("New order's total: " + updated.total());

        // Sharing an immutable object across threads needs NO synchronization at all
        Runnable reader = () -> System.out.println(Thread.currentThread().getName()
            + " safely reads: " + order.id() + ", " + order.total());
        Thread t1 = new Thread(reader), t2 = new Thread(reader);
        t1.start(); t2.start(); t1.join(); t2.join();
    }
}
```


Output

```
Order items unaffected by later external mutation: [Laptop, Mouse]
Original total unchanged: 55000.0
New order's total: 55500.0
Thread-0 safely reads: ORD1, 55000.0
Thread-1 safely reads: ORD1, 55000.0
```

Thread-Safe Class Design Checklist

1. **All fields `final`**, set only in the constructor.
2. **The class itself `final`** (or otherwise ensure no subclass can break invariants) — an unsafe subclass could otherwise expose mutable state or violate the immutability contract.
3. **Defensive copies** of any mutable input (like `List.copyOf(items)` above) — without this, external code retaining a reference to the original mutable collection could still mutate the "immutable" object's internal state through that back door.
4. **Never return internal mutable references directly** from accessors — always return an immutable view/copy, or ensure the internal reference is already genuinely immutable.
5. **No setters** — "modification" methods return a new instance instead of mutating (`withAdditionalTotal()` above).

Internal Working

- The reason immutable objects need **zero synchronization** connects directly to Ch.6's Java Memory Model: as long as an immutable object is **safely published** (e.g., via a `final` field, or a properly synchronized/volatile handoff, or simply constructed and handed off before any other thread could see a partially-constructed reference), every field is guaranteed visible and consistent to every thread that subsequently reads it — there's no compound state to race on, because nothing ever changes after construction.
- The **defensive copy** step is the single most commonly *forgotten* part of building a genuinely immutable class — simply making fields `final` is not enough if a `final` field references a *mutable* object (like a `List`) that the constructor didn't copy; Book 01, Ch.9's "final reference, mutable content" trap applies directly here, with real concurrency consequences on top of the correctness consequences already discussed there.

Real-World Example

Telugu: DTOs (Book 07, Ch.13's records), configuration objects, value objects (Money, Address) — వీటిని immutable నా design చేయడం, multi-threaded services లో వాటిని ఎలాంటి lock లేకుండా safely share చేయడానికి అనుమతిస్తుంది — production concurrency bugs తగ్గించడానికి ఇది అత్యంత effective, low-effort technique. English: Designing DTOs (Book 07, Ch.13's records are immutable by default), configuration objects, and value objects (Money, Address) as immutable is, in practice, the single highest-leverage, lowest-effort technique for reducing concurrency bugs in real multi-threaded services — they can be freely shared across threads with zero locking overhead at all.

Interview Answer

"Immutable objects are inherently thread-safe with zero synchronization, since there's no mutable state to race on after construction. A genuinely thread-safe immutable class needs final fields set only in the constructor, no setters (modifications return new instances), defensive copies of any mutable input, and ideally a final class to prevent subclasses from breaking the contract. This is the single most effective, lowest-overhead technique for avoiding concurrency bugs in practice."

Cross Questions

- Q: Is making all fields `final` sufficient for true immutability? → A: No — if a `final` field references a mutable object (like a `List` or array) without a defensive copy, external code can still mutate that referenced object's internal state, breaking immutability despite the field itself never being reassigned.
- Q: Why does an immutable object need no synchronization at all when shared across threads? → A: There's no compound or evolving state for concurrent access to race on — every field is set once, before the object is shared, and connects to the JMM's safe-publication guarantees (Ch.6).
- Q: Why should a genuinely immutable class be declared `final`? → A: To prevent a subclass from adding mutable state or overriding methods in a way that violates the immutability contract the base class was designed to guarantee.

Tricky Questions

- Q: If an immutable class's constructor doesn't properly synchronize/safely-publish the object (e.g., leaks `this` to another thread before construction finishes), can thread-safety still be broken? → A: Yes — this is a genuine, subtle risk called "unsafe publication"; even a technically-immutable object can appear partially-constructed to another thread if a reference escapes before the constructor completes (e.g., registering `this` in a listener list from within the constructor) — proper safe publication (final fields, or handing off the fully-constructed reference only after construction) is required.
- Q: Does returning `List.copyOf(items)` from an accessor protect against ALL mutation risks? → A: It protects against the caller mutating the *returned* list affecting the object's internal state, and vice versa — but if `items` itself contained mutable elements (e.g., `List<StringBuilder>`), those individual elements could still be mutated through the returned references; true deep immutability requires every contained element to also be immutable.

Part 1 Mini Project: Thread-Safe Bank Account Simulator

Goal: Combine every concept from Part 1 into one realistic concurrent system.

Requirements:

1. An immutable `TransactionRecord` (Ch.10) logging every operation (id, type, amount, timestamp via `java.time`, Book 07 Ch.10).
2. A `BankAccount` class with `deposit()` / `withdraw()` protected by a `ReentrantLock` (Ch.8) — NOT synchronized, to practice explicit lock discipline (always unlock in `finally`).

3. An `AtomicLong` (Ch.7) tracking total transaction count across all accounts, incremented on every operation.
4. A `transferMoney(BankAccount from, BankAccount to, double amount)` method that avoids deadlock (Ch.9) using **consistent lock ordering** by account ID, when locking both accounts.
5. A `volatile boolean systemOpen` flag (Ch.5) that, when set to `false`, causes all in-progress and new operations to reject further transactions gracefully.
6. Run a simulation: 10 threads performing 1,000 random deposits/withdrawals/transfers across 5 accounts concurrently, then verify: (a) total money in the system is conserved (no lost updates, Ch.3), (b) no deadlock occurred, (c) the atomic transaction counter matches the expected total.

Concepts Reinforced: Thread lifecycle (Ch.1) · Runnable/Callable (Ch.2) · race conditions (Ch.3) · locks and synchronized (Ch.4, 8) · volatile (Ch.5) · JMM/happens-before (Ch.6) · atomics (Ch.7) · deadlock avoidance (Ch.9) · immutable design (Ch.10).

Stretch Goal: Add a `ReadWriteLock`-protected (Ch.8) shared "interest rate" configuration value, read frequently by all accounts but updated rarely by an admin thread.



PART 1 FINAL REVISION NOTES

- **Process vs Thread:** threads share heap/method area, have private stack/PC register (Book 03).
 - **Thread lifecycle:** NEW → RUNNABLE ⇌ BLOCKED/WAITING/TIMED_WAITING → TERMINATED; calling `run()` directly ≠ concurrency.
 - **Runnable vs Callable:** no-return/no-checked-exception vs returns-value/can-throw, retrieved via `Future.get()` (blocks).
 - **Race conditions:** shared mutable state + non-atomic compound ops (`count++`) → lost updates; non-deterministic, timing-dependent.
 - **synchronized** : intrinsic monitor, mutual exclusion, auto-release (even on exception), reentrant; prefer a private lock object over `this` .
 - **volatile** : fixes visibility + establishes happens-before ordering; does NOT fix compound-operation atomicity.
 - **JMM/happens-before:** program order, lock unlock→lock, volatile write→read, thread start→body→join; transitive — one flag can publish a whole batch of writes.
 - **Atomics/CAS:** lock-free, optimistic concurrency; genuinely atomic compound ops; watch for the ABA problem; can underperform locks under very high contention.
 - **Locks:** `ReentrantLock` (manual unlock, timed/interruptible, fairness), `ReadWriteLock` (concurrent readers), `StampedLock` (optimistic reads).
 - **Deadlock/livelock/starvation:** 4 Coffman conditions; fix deadlock via consistent lock ordering or `tryLock` + backoff.
 - **Immutability:** the highest-leverage thread-safety technique — final fields, defensive copies, no setters, final class; zero synchronization needed once safely published.
-



PART 1 CHEAT SHEET

Thread states: NEW -> RUNNABLE <-> BLOCKED/WAITING/TIMED_WAITING -> TERMINATED
Runnable: no return, no checked throws | Callable<V>: returns V, can throw | Future.get()
BLOCKS
Race condition: shared mutable state + non-atomic compound op (count++ = read+incr+write, 3 steps)
synchronized: intrinsic monitor, mutual exclusion, AUTO-release, reentrant; prefer private lock object over 'this'
volatile: visibility + ordering (happens-before) ONLY - NOT atomicity for compound ops
Happens-before edges: program order | unlock->lock (same monitor) | volatile write->read | thread start->body->join
Atomic classes: CAS-based, lock-free, optimistic; watch ABA problem;
AtomicInteger/Long/Reference
ReentrantLock: manual unlock (ALWAYS in finally!), tryLock(timeout), fairness | ReadWriteLock: N readers XOR 1 writer
StampedLock: + optimistic read (tryOptimisticRead + validate)
Deadlock: circular wait, 4 Coffman conditions (mutex, hold-wait, no-preempt, circular-wait) - fix: consistent lock order
Livelock: active but no progress | Starvation: perpetually denied resource
Immutable design: final fields + final class + defensive copies + no setters = thread-safe with ZERO sync needed



PART 1 INTERVIEW QUESTION BANK

Beginner

1. What is the difference between a process and a thread?
2. What is the difference between `Runnable` and `Callable`?
3. What is a race condition? Give a simple example.
4. What does `synchronized` guarantee?
5. What is the difference between `volatile` and `synchronized`?

Intermediate 6. Why is `count++` not atomic, even though it looks like a single operation? 7. Explain the happens-before relationship with at least 2 concrete edges. 8. What is Compare-And-Swap, and how do atomic classes use it? 9. Why must `ReentrantLock.unlock()` always be called in a `finally` block? 10. What are the 4 Coffman conditions for deadlock?

Advanced 11. Explain why `volatile` fixes visibility but not atomicity, with a code example proving it. 12. Explain the "volatile ready flag" pattern and why happens-before's transitivity makes it work. 13. Explain the ABA problem and how `AtomicStampedReference` addresses it. 14. Compare `ReentrantLock`, `ReadWriteLock`, and `StampedLock` — when would you choose each? 15. Design a deadlock-free bank transfer method between two accounts.

Senior/Architect 16. Diagnose a production race condition bug that only appears under high load and never in testing — describe your investigation process. 17. Design a thread-safe, immutable configuration object safely shared across a high-throughput multi-threaded service. 18. Explain how you'd choose between `synchronized`, `ReentrantLock`, and atomic classes for a given shared-counter requirement, covering contention level trade-offs. 19. Review a codebase with inconsistent lock-acquisition ordering across multiple methods — describe the deadlock risk and your remediation plan. 20. Explain, end-to-end, why immutability is considered the highest-leverage thread-safety technique, and what "unsafe publication" risk remains even for immutable objects.

(Full short/professional/deep-senior answer scaffolding — combined with Part 2's questions — lives in Book 23, Java Interview Master Book.)



PART 1 CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining each concurrency mechanic out loud in Telugu. **L3 — Advanced:** Reproduce and fix the Ch.3 race condition using 3 different techniques (synchronized, atomic, ReentrantLock) and compare. **L4 — Interview:** Answer all 20 Part 1 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 10 mini project fully, including the ReadWriteLock stretch goal. **L6 — Mastery:** Teach Chapters 3 (race conditions), 6 (JMM/happens-before), and 9 (deadlock) out loud, from memory, using fresh examples.



PART 1 — ONE-DAY REVISION PLAN (≈5 hours)

Time	Focus
0:00–0:30	Ch.1–2: Thread lifecycle, Runnable/Callable/Future
0:30–1:00	Ch.3: Race conditions — reproduce the lost-update bug
1:00–1:30	Ch.4: synchronized deep dive
1:30–1:45	Break
1:45–2:15	Ch.5: volatile — memorize what it does and does NOT guarantee
2:15–3:00	Ch.6: JMM/happens-before — highest-density block, re-read twice
3:00–3:30	Ch.7: Atomic classes & CAS
3:30–4:00	Ch.8: Locks comparison table
4:00–4:30	Ch.9: Deadlock/livelock/starvation
4:30–5:00	Ch.10 + Interview Bank — immutability checklist, answer questions from memory

PART 1 MASTERY CHECKLIST

- ☐ I can draw the full thread lifecycle diagram from memory.
- ☐ I can explain why `count++` is not atomic and reproduce the resulting bug.
- ☐ I can explain `synchronized`'s guarantees, including reentrancy and automatic release.
- ☐ I can explain exactly what `volatile` does and does not guarantee.
- ☐ I can state the key happens-before edges and explain their transitivity.
- ☐ I can explain CAS and the ABA problem.
- ☐ I can compare `synchronized`, `ReentrantLock`, `ReadWriteLock`, and `StampedLock`.
- ☐ I can state the 4 Coffman conditions and the standard deadlock-avoidance fix.
- ☐ I can design a genuinely thread-safe immutable class, including defensive copies.
- ☐ I built the Thread-Safe Bank Account Simulator mini project, including the stretch goal.
- ☐ I answered the full Part 1 Interview Question Bank confidently in both Telugu and English.

Once every box is checked, continue to [08_Java_Multithreading_Concurrency_Part_2.md](#) — ExecutorService, thread pools, CountdownLatch/CyclicBarrier/Semaphore, BlockingQueue, ConcurrentHashMap internals, CompletableFuture in depth, Virtual Threads, and production concurrency patterns.